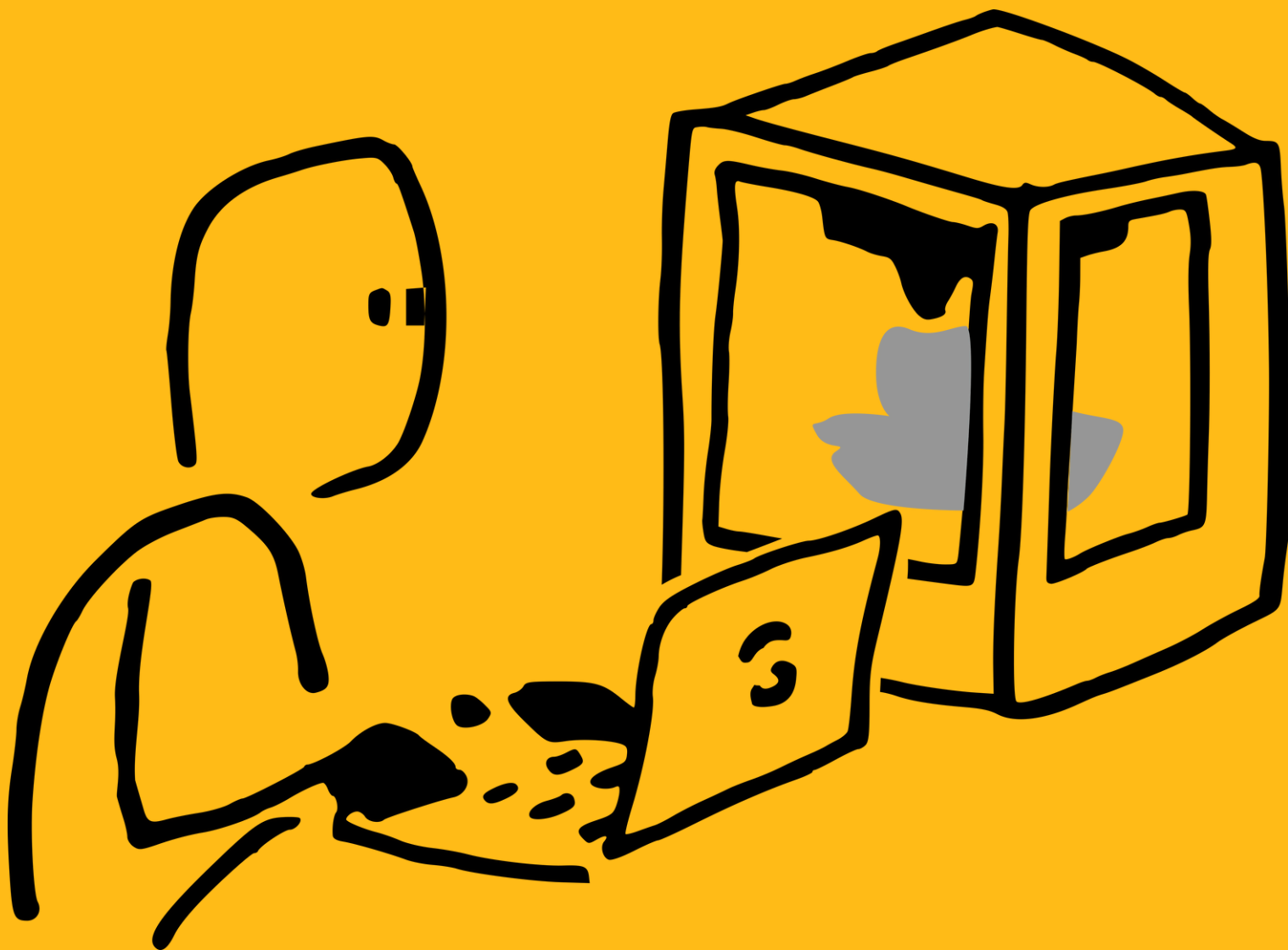




Prototype

All things you should know about Prototype.



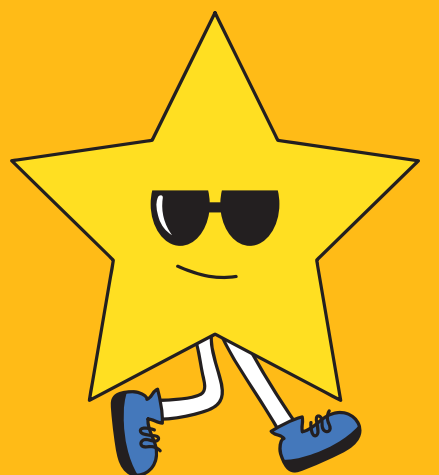
What is Prototype?

It belongs to the creational category.

Determine the kinds of object to create using a prototypical instance and create a new object by copying this prototype.



Shirin Monzavi



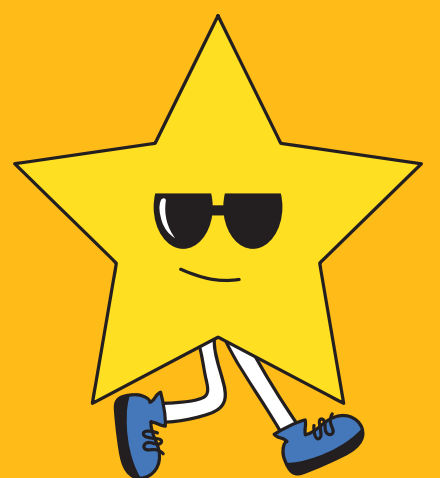


Problem

Let's imagine there is a general framework for graphical editors. It provides an abstract class for graphical components such as notes and staves. In addition, it provides an abstract Tool class for defining tools like those in the palette. One predefined subclass of Tool is GraphicalTool, which creates instances of graphical objects and adds them to a document.



Shirin Monzavi



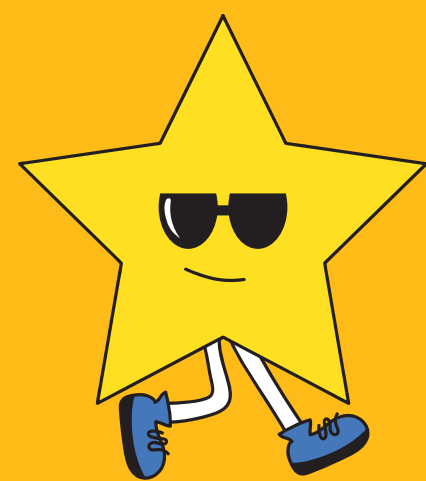


Problem

We want to create an editor for music score by customizing this framework. Music score framework contains notes and rests objects. This editor may have a palette of tools for adding these music object to the score. The problem with the Graphical tool is the **classes of notes and staves are specific to our program**, but **the Graphic tool does not know how to create instances of them**. One approach might be, subclass Graphic tool for each kind of object in order to create instance and add them to the score, but this would **lead to a lot of subclasses** that are only different in the kind of music object. So how can we configure the graphical framework in a way that Graphical Tool accepts the instance of Graphic class and create them?



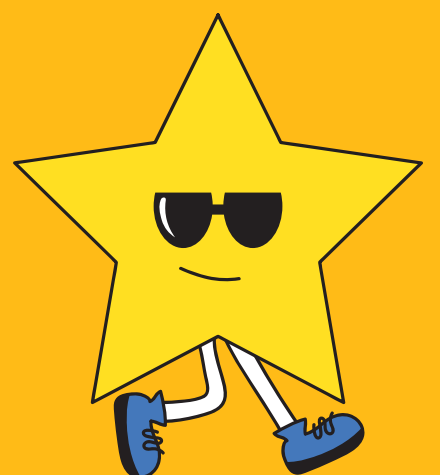
Shirin Monzavi



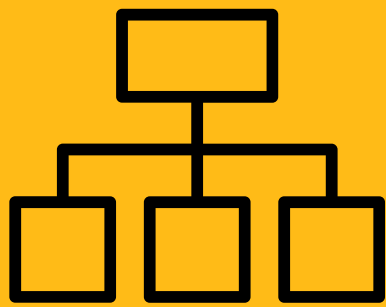


Solution

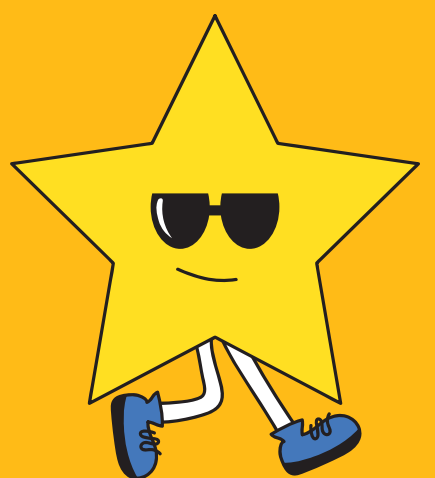
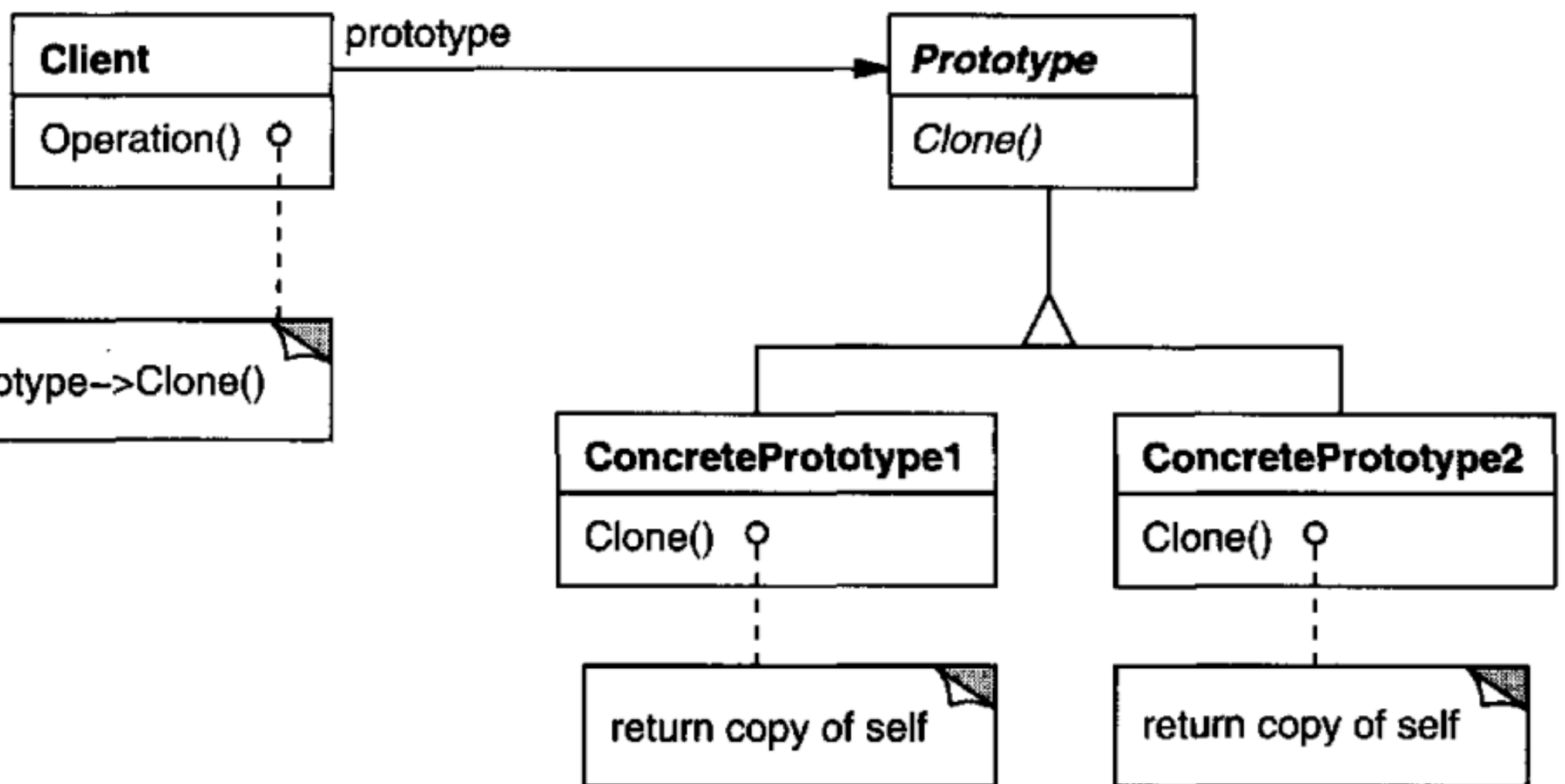
If all subclasses of Graphic support a **Clone operation**, then the Graphical Tool which is parameterized with Graphic class can clone any kind of Graphic then add them to document.



Shirin Monzavi



Structure





Usability

Sometimes classes to instantiate are specified at run-time. for instance, dynamic loading.

When applying a factory pattern, you might end up with a class hierarchy for the products and another hierarchy for the factories. The Prototype pattern eliminates the need for a separate factory hierarchy by directly cloning prototypes.

If a class have several states, it may be more efficient to create a set of prototype objects with different states, and clone them as needed, rather than constructing new instances each time with appropriate states.



Shirin Monzavi

follow

↕ Repost